

Rapport hebdomadaire

ProtoVA1 : remise en service — ProtoVA2 : démo 5G rétablie

Douae SEBTI — Projet de fin d'études, ALTEN Labs (ProtoVA)

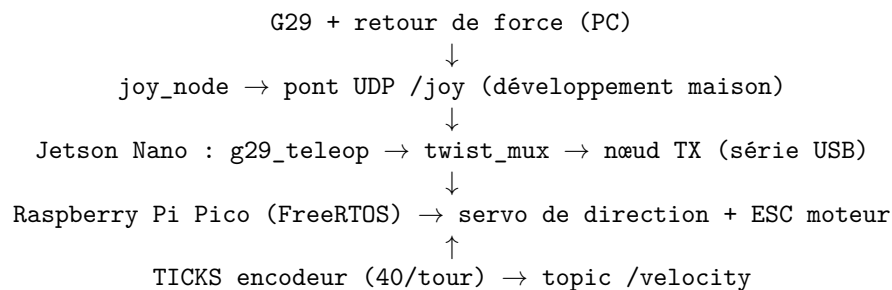
Semaine du 3 au 6 août 2026

1 Objectif de la semaine

Rendre **ProtoVA1** (Jetson Nano, ROS 2 Foxy) de nouveau **pleinement téléopérable** au volant Logitech G29 avec retour de force, en intégrant le nouvel encodeur et son étage de filtrage — première étape vers la démonstration finale : **deux plateformes opérationnelles échangeant des messages V2V (DENM)**.

Objectif atteint : la voiture a roulé au sol, téléopérée au G29 avec retour de force.

2 Architecture mise en place



3 Travaux réalisés

3.1 Micrologiciel Raspberry Pi Pico (réécrit, compilé, flashé)

- **FreeRTOS**, trois tâches : réception série (CTRL, angle, vitesse), contrôle servo + ESC, encodeur (TICKS, n à 50 Hz) ;
- **recalibration du neutre ESC** : 1400 → 1500 μs — bug critique : à 1400 μs , la voiture partait en marche arrière rapide sur la commande « stop » ;
- **armement automatique de l'ESC** : le signal neutre est désormais émis dès la mise sous tension (auparavant, le micrologiciel attendait l'ouverture du port série avant d'émettre le moindre signal, ce qui obligeait à débrancher puis rebrancher l'alimentation à chaque session).

3.2 Encodeur KY-040 et conditionnement du signal

- chaîne validée : encodeur → filtre RC → bascule de Schmitt **74HC14** → Pico (élimination des rebonds mécaniques) ;

- étude des décodages possibles ($\times 1 = 20$, $\times 2 = 40$, $\times 4 = 80$ ticks/tour) → **décodage $\times 2$ retenu : 40 ticks/tour** ;
- paramètres d'odométrie mis en cohérence côté ROS (`ticks_per_rev = 40` dans les nœuds TX et `ackermann_odom`).

3.3 Réseau : pont UDP pour le topic /joy

- diagnostic : le partage de connexion utilisé **bloque les données DDS** entre machines (la découverte passe, les données non) — vérifié objectivement, l'UDP brut passant dans les deux sens ;
- développement d'un **pont UDP unicast** dédié : le PC envoie /joy sur le port 9001, la Jetson le republie localement → interopérabilité Humble (PC) ↔ Foxy (Jetson) rétablie.

3.4 Pile logicielle Jetson

- `twist_mux` installé et configuré (priorités : AEB 255 > PS4 60 > G29 30) — prêt pour l'arrivée d'un freinage d'urgence coopératif ;
- script de lancement unique (`launch_teleop_p1.sh`) + service systemd `p1teleop` résilient aux coupures SSH ;
- *runbook* complet documenté (flash → armement → lancement → conduite → diagnostic).

3.5 Premier roulage au sol et itération de réglages

Constat	Cause	Correction
Retards à l'accélération	Consigne minimale dans la zone morte de l'ESC	Minimum relevé au-dessus de la zone morte
Vitesse bridée	Pédale à fond = 20 % de puissance seulement	Plage élargie à 35 % (ajustable)
Manque de précision	Course de pédale écrasée sur une plage étroite	Plage élargie → meilleure résolution
ESC à réarmer en débranchant	Micrologiciel muet avant ouverture du port série	Neutre émis dès la mise sous tension

TABLE 1 – Constats du premier roulage et corrections apportées.

4 ProtoVA2 : remise en route de la démonstration de téléopération 5G

La démonstration 5G (téléopération G29 sur ROS 2 via **Zenoh**, PC hub ↔ véhicule en 172.16.48.7/6) était inopérante depuis le re-flashage de la Jetson Orin. Diagnostic et remise en route complète :

4.1 Cause racine identifiée : la mémoire partagée de Zenoh

Les nœuds ROS du véhicule mouraient tous à l'initialisation (`Failed to create POSIX SHM provider, OS error 12`) alors que le routeur Zenoh, lui, survivait — d'où l'impression trompeuse d'un lien « cassé ». En cause : le paquet `rmw_zenoh_cpp` réinstallé après le re-flashage (build plus récent que celui du PC) tente d'allouer un segment de **mémoire partagée** (SHM) au démarrage de chaque nœud et échoue, alors même que la mémoire disponible était suffisante (3,8 Go libres). **Correctif** : désactivation de la SHM dans la configuration de session Zenoh du véhicule

(`transport: { shared_memory: { enabled: false } }` dans `zenoh_session_car.json5`) — sans perte fonctionnelle, la SHM n'optimisant que les échanges *intra-machine*, inutile pour un lien 5G.

4.2 Remises d'aplomb annexes

- adresse fantôme /30 du DHCP purgée côté véhicule (re-passage en /28, piège récurrent de cette installation);
- route par défaut 5G supprimée côté PC (le trafic internet repassait par le lien cellulaire);
- service `car5g` réactivé au démarrage.

4.3 Validation de bout en bout

Topics du véhicule visibles depuis le PC à travers la 5G; /joy (volant) transmis PC → véhicule à ~16 Hz; /cmd_vel produit à bord et transmis au Pico; tableau de bord de latence en direct accessible à travers le lien (<http://172.16.48.6:8088>); retour caméra dans `rqt` via le flux **comprimé** (obligatoire : le flux brut 720p (~600 Mbps) excède la capacité montante du lien 5G (~70 Mbps), le JPEG comprimé y tient à 30 im/s). *Runbook* 5G complet documenté (lancement en deux terminaux + arbre de diagnostic).

5 ProtoVA2 : cartographie SLAM en direct et exploration autonome

Travail réalisé sur **ProtoVA2** (Jetson Orin, ROS 2 Humble, liaison Wi-Fi FastDDS) : le véhicule cartographie son environnement en temps réel *tout en se conduisant seul* — il avance vers les passages libres, freine, recule en braquant pour se dégager et repart ailleurs, à la manière d'un conducteur réel.

5.1 Chaîne de localisation et cartographie (validée par la mesure)

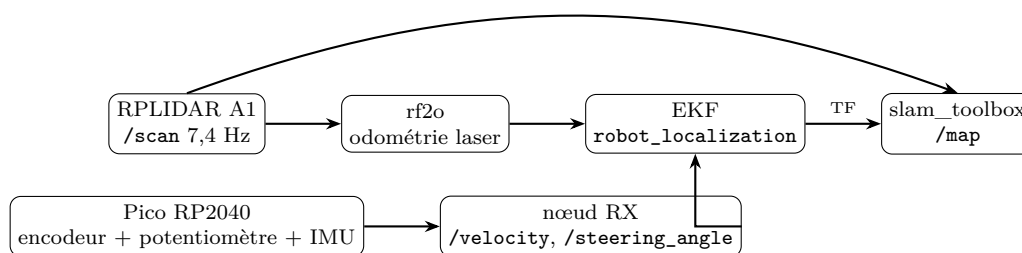


FIGURE 1 – Chaîne de localisation ProtoVA2 : trois odométries (laser, encodeur, IMU) fusionnées par un filtre de Kalman étendu, alimentant la cartographie.

Chaque étage a été **vérifié par la mesure** avant de rouler (méthode : isoler chaque défaillance au lieu de deviner) :

Vérification	Mesure	Verdict
Cadence lidar /scan	7,4 Hz	nominale (RPLIDAR A1)
Sortie EKF /odometry/filtered	7,7 Hz	fonctionnelle (30 Hz configurés, à optimiser)
Dérive à l'arrêt (20 s)	2,6 mm, 0°	pas de biais capteur
Arbre TF map→odom→base_link	complet	cohérent

TABLE 2 – Diagnostic de la chaîne de localisation avant roulage.

5.2 Visualisation en direct (contournement d'un bug RViz)

Le tracé de la carte dans RViz 2 souffre d'un bug de *shader* connu (carte rendue en jaune uni à chaque mise à jour, y compris en rendu logiciel). Plutôt que de subir, un **visualiseur web dédié** a été développé (`map_live.py`) : il s'abonne à `/map` et à la TF, dessine la carte (PNG) avec la pose du véhicule, et la sert sur `http://localhost:8090` avec rafraîchissement automatique (1 s) — robuste, sans dépendance graphique. RViz reste utilisé pour le nuage lidar, les axes TF et l'odométrie.

5.3 Exploration autonome : machine à états « follow-the-gap »

Le nœud `roamer.py` (écrit cette semaine) conduit le véhicule sans carte préalable : il analyse chaque tour de lidar, vise le passage libre le plus large, et gère les impasses par une manœuvre de dégagement complète.

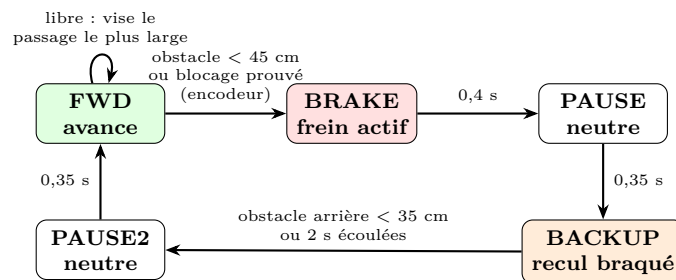


FIGURE 2 – Machine à états du `roamer` : la séquence frein → neutre → recul reproduit le « double appui » exigé par l'ESC pour enclencher la marche arrière ; le recul se fait en contre-braquage pour pointer le nez vers le passage visé, l'arrière étant surveillé au lidar (360°).

Points clés du comportement :

- **préférence à l'avant** : le recul n'est déclenché qu'en dernier recours (obstacle réellement proche, ou *blocage prouvé* : gaz commandé mais encodeur immobile pendant 2 s — détection par **proprioception**, robuste aux obstacles invisibles du lidar) ;
- **sécurité en couches** : le `roamer` publie en priorité *basse* (10) dans `twist_mux` — le volant G29 (30) et l'AEB (255) le dominant toujours ; un script d'arrêt d'urgence (`stop_all.sh`) tue tous les nœuds *et* remet la Pico au neutre (elle mémorise la dernière commande reçue).

5.4 Anomalies débusquées pendant l'intégration

6 Livrables (dépôt GitHub)

-
- `protoval/pico_car_freertos.ino` — micrologiciel final ;
 - `protoval/joy_udp_bridge_tx.py` / `joy_udp_bridge_rx.py` — pont /joy ;
 - `protoval/launch_teleop_p1.sh` — pile complète ;
 - `docs/protoval_teleop_runbook.md` — procédure reproductible ;
 - `docs/demo5g_runbook.md` — *runbook* de la démo 5G (lancement + diagnostic, à jour du correctif SHM) ;
 - `roamer.py` — exploration autonome follow-the-gap (machine à états, proprioception encodeur) ;
 - `map_live.py` — visualiseur web de la carte SLAM en direct ;
 - `stop_all.sh` — arrêt d'urgence logiciel (kill global + neutre Pico) ;
 - `nav_slam.rviz` — configuration RViz lidar/TF/odométrie.

Anomalie	Correction
Le nœud RX (encodeur/potentiomètre) crashait au démarrage depuis le re-flashage (<code>np.float</code> supprimé du nouveau numpy, utilisé par le vieux <code>transforms3d</code>) → véhicule <i>aveugle sur son propre corps</i> , fausses détections de blocage en boucle	Mise à jour de <code>transforms3d</code> (0.4.2); <code>/velocity</code> et <code>/steering_angle</code> publient à nouveau
Deux piles de conduite lancées en double → deux écrivains sur le port série de la Pico, commandes entremêlées, servo inerte	Nettoyage + règle « une seule pile à la fois » documentée
Le micrologiciel Pico <i>mémorise</i> la dernière commande : si le logiciel meurt en roulant, le véhicule continue indéfiniment	<code>stop_all.sh</code> (kill global + neutre série); correctif définitif identifié : <i>watchdog</i> micrologiciel (neutre auto après 0,5 s sans commande) — planifié
Consignes de gaz faibles sous le seuil de roulement (batterie faible) → le logiciel « croit » avancer	Vitesses relevées + détection de blocage par encodeur

TABLE 3 – Anomalies identifiées et corrigées pendant l'intégration.

7 Prochaines étapes : vers la démonstration à deux véhicules

1. **ProtoVA2** : investigation du bug « avance sans accélérateur » (boîte noire), calibrations ;
2. émission d'un **DENM au freinage AEB** côté ProtoVA2 (Vanetza déjà opérationnel) ;
3. portage V2V sur ProtoVA1 (compilation de Vanetza sur la Jetson Nano + nœud CAM/DENM léger) ;
4. **démonstration finale** : ProtoVA2 freine (AEB) → DENM → ProtoVA1 réagit — sécurité coopérative entre deux véhicules réels.